From Baseline to Production-Minded

Exactly what improved between *german_credit_risk.ipynb* (submitted) and *german_credit_risk_bo_executed.ipynb* (improved), with the theory, mathematics and statistics behind each change

Prepared for	Equal Experts – Data Scientist pair-programming session (follow-up to the offline case study)
Dataset / task	Kaggle Weka German Credit – 1,000 applicants, 300 bad / 700 good; *bad* is the positive class
Evidence base	Every result quoted below is read directly from the executed outputs of the two notebooks. Items marked † are my own supplementary calculations from those reported numbers or from synthetic checks of the theory.

1. Executive summary

The improved notebook keeps the original problem framing, split (stratified 80/20, seed 42), leakage-safe pipelines and the logistic-regression-versus-random-forest comparison. It then adds **eight** things. One of them changes the model (Bayesian hyperparameter optimisation); the other seven change how much the results can be *trusted, explained and operated*. An honest reading of the numbers matters, so the headline table separates what moved from what did not.

Metric (random forest unless stated)	Original	Improved	Reading
Training 5-fold CV ROC-AUC – logistic regression	0.7687	**0.7838**	**+0.0151**: tuning L2 strength (C 1.0 → 0.042) helped the baseline
Training 5-fold CV ROC-AUC – random forest	0.7974	0.7970	Flat (−0.0004): forest is insensitive to its hyperparameters here
Holdout ROC-AUC	0.801	0.801	Unchanged
Holdout average precision	0.650	0.645	Within noise
Holdout balanced accuracy @0.5	0.702	0.689	Slightly lower (bad-class recall 0.58 → 0.55)
Holdout Brier score	0.169	0.169	Unchanged
Holdout AUC uncertainty	not reported	**95% CI 0.730–0.864**	New: quantifies what a 200-row, 60-default holdout can say

The key message

Bayesian optimisation raised the *baseline* but not the *champion*. The champion's holdout performance is statistically unchanged. The real improvement is in **evidence quality**: confidence intervals, a corrected model comparison, calibration awareness, feature-rank stability, a fairness audit, a guarded scoring interface and SHAP explanations. That is the right story for a pair-programming discussion because it shows I can tell a genuine gain from a noisy one.

2. Change map: where each improvement lives

#	Improvement	Original notebook	Improved notebook (cells)	Underpinning
1	Bayesian hyperparameter optimisation (GP + expected improvement)	Fixed settings: LR C=1, balanced; RF 500 trees, leaf 3	11–15, 17 (5 random + 10 BO trials per model)	Gaussian processes, Matern-5/2 kernel, EI acquisition, MAP view of L2
2	Bootstrap uncertainty on holdout AUC	Point estimate only	21 (part 1)	Mann–Whitney form of AUC, stratified percentile bootstrap
3	Repeated 5×5 CV, paired model comparison	Single 5-fold CV	21 (part 2)	Dependence between resamples, paired t, Nadeau–Bengio
4	Class-weight / calibration check	Balanced weights assumed	21 (part 3)	Prior-shift (odds scaling), Brier score, AUC invariance
5	Feature-rank stability across folds	One permutation ranking	21 (part 4)	Permutation importance, selection frequency

#	Improvement	Original notebook	Improved notebook (cells)	Underpinning
6	Age-group fairness and proxy audit	None	21 (part 5)	False-positive-rate parity, Wilson interval, proxy AUC
7	Strict prediction-time validation layer	None	21 (part 6)	Data contracts, training/serving skew, fail-fast design
8	SHAP explanations, aggregated to raw fields	None	22–34	Shapley axioms, TreeSHAP, additivity under grouping

Smaller hygiene changes. Data loading became a function that raises `ValueError` instead of relying on `assert` (asserts vanish under `python -O`); a single `make_pipeline()` factory builds every model; EDA expanded to numeric overlays, row-normalised category mixes and a split-balance check (cells 7–10); the held-out evaluation now also prints the selected hyperparameters.

3. Improvement 1 – Bayesian hyperparameter optimisation

3.1 The problem being solved

Choosing hyperparameters θ (for logistic regression: the inverse regularisation strength C and the class-weight option; for the forest: depth, leaf size, feature subsampling and so on) is a black-box optimisation problem. The objective is the mean stratified-CV ROC-AUC on the training split:

$$f(\theta) = \frac{1}{K}\sum_{k=1}^{K} \mathrm{AUC}_k(\theta), \qquad \theta^{*} = \arg\max_{\theta \in \Theta} f(\theta)$$

Each evaluation of f costs a full 5-fold refit, the function has no gradient, and it is *noisy* because CV scores vary with the fold split. Grid search wastes evaluations in unpromising regions; random search ignores what it has already learned. Bayesian optimisation (BO) instead keeps a probabilistic model of f and spends each new evaluation where it is expected to be most useful.

3.2 Surrogate model: a Gaussian process

The notebook places a Gaussian-process prior on f with kernel `ConstantKernel * Matern(nu=2.5) + WhiteKernel`:

$$f \sim \mathcal{GP}(m,\, k), \qquad k(r) = \sigma^2\left(1 + \frac{\sqrt{5}\, r}{\ell} + \frac{5r^2}{3\ell^2}\right)\exp\left(-\frac{\sqrt{5}\, r}{\ell}\right) + \sigma_n^2\, \mathbf{1}[r = 0]$$

The Matérn-5/2 kernel assumes f is twice differentiable but not infinitely smooth, which suits ML validation curves better than the squared-exponential kernel. The `WhiteKernel` term σ_n^2 models observation noise, which is the right choice for noisy CV scores. `normalize_y=True` standardises the scores so the prior mean/variance are sensible. Given t observations $\mathbf{y}$ at encoded points X, the posterior at a new point x is Gaussian with

$$\mu(x) = \mathbf{k}_*^{\top}\left(K + \sigma_n^2 I\right)^{-1}\mathbf{y}, \qquad \sigma^2(x) = k(x,x) - \mathbf{k}_*^{\top}\left(K + \sigma_n^2 I\right)^{-1}\mathbf{k}_*$$

Hyperparameters are encoded to [0,1]: log-uniform for C (`log_float`), min–max for integers, one-hot for categoricals (`encode_params`). Working in log C matters because AUC responds to orders of magnitude of regularisation, not to additive steps.

3.3 Acquisition function: expected improvement

With incumbent best score f^{+} and exploration margin ξ (the notebook uses ξ = 0.01), expected improvement has a closed form:

$$\mathrm{EI}(x) = \mathbb{E}[\max\{f(x) - f^{+} - \xi,\, 0\}] = (\mu - f^{+} - \xi)\,\Phi(z) + \sigma\,\phi(z), \qquad z = \frac{\mu - f^{+} - \xi}{\sigma}$$

Derivation. Write f(x) = μ + σε with ε ~ N(0,1) and let c = f^{+} + ξ − μ. Then EI = σ $\int_{c/\sigma}^{\infty}$ (ε − c/σ) ϕ(ε) dε. Using $\int_a^{\infty}$ εϕ(ε)dε = ϕ(a) and $\int_a^{\infty}$ ϕ = 1 − Φ(a) gives σ[ϕ(a) − a(1 − Φ(a))] with a = c/σ = −z, which equals the

expression above. The first term rewards a high predicted mean (*exploitation*), the second rewards high uncertainty (*exploration*). † I confirmed numerically that the closed form matches a 2-million-draw Monte-Carlo estimate (4.553e-4 vs 4.564e-4 for μ=0.78, σ=0.01, f^{+}=0.783).

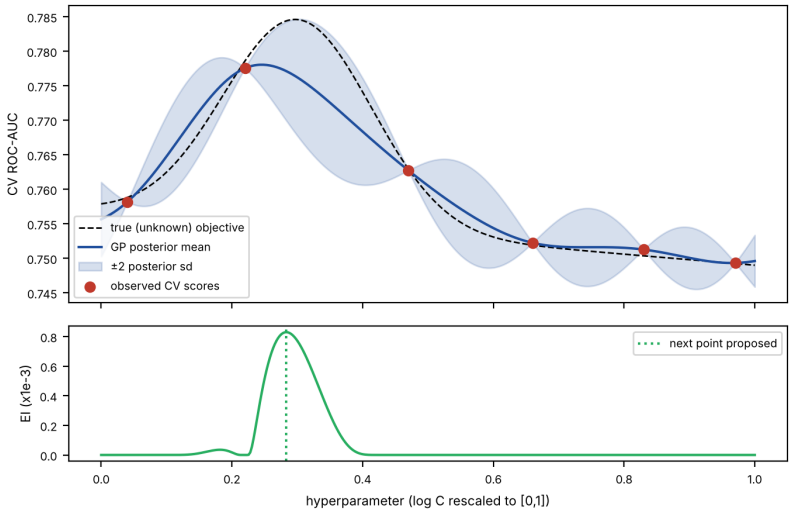


Figure 1 – † Schematic of one BO step on a synthetic 1-D objective: the GP posterior (top) and the EI surface (bottom) that selects the next hyperparameter.

3.4 The loop as implemented

```
def bayesian_optimize(name, space, prep, X_train, y_train, cv, n_initial=5, n_iter=10, seed=RANDOM_STATE):
    rng = np.random.default_rng(seed); params_seen, scores, points, history = [], [], [], []
    for trial in range(n_initial + n_iter):
        if trial < n_initial:                    # 1) random warm start
            params = sample_params(space, rng)
        else:
            gp = GaussianProcessRegressor(
                kernel=ConstantKernel(1.0) * Matern(nu=2.5) + WhiteKernel(),
                normalize_y=True, random_state=seed + trial)
            gp.fit(np.vstack(points), np.asarray(scores))            # 2) fit surrogate
            candidates = [sample_params(space, rng) for _ in range(256)]
            cand_pts = np.vstack([encode_params(p, space) for p in candidates])
            mean, std = gp.predict(cand_pts, return_std=True)
            improvement = mean - max(scores) - 0.01                  # f(x) - f+ - xi
            z = np.divide(improvement, std, out=np.zeros_like(std), where=std > 1e-12)
            ei = improvement * norm.cdf(z) + std * norm.pdf(z)       # 3) closed-form EI
            for i, c in enumerate(candidates):
                if c in params_seen: ei[i] = -np.inf                 # never repeat a point
            params = candidates[int(np.argmax(ei))]                  # 4) maximise EI over 256 draws
        score = cross_val_score(make_pipeline(name, params, prep), X_train, y_train,
                                cv=cv, scoring="roc_auc", n_jobs=1).mean()   # 5) evaluate f(theta)
        params_seen.append(params); scores.append(float(score)); points.append(encode_params(params, space))
```

Listing 1 – Core of the BO loop (cell 12), lightly abridged. Preprocessing sits inside the pipeline, so every CV fit re-learns scalers and encoders on training folds only.

Two implementation choices are worth being able to defend. First, EI is maximised over 256 random candidates rather than by gradient ascent; that is cheap and robust for a 2–6-dimensional mixed-type space. Second, the `-inf` mask prevents re-proposing an identical configuration, which would otherwise waste a trial when the objective is noisy.

3.5 What the optimisation found

Model	Original (fixed)	Best random warm-up (trials 1–5)	Best after BO (15 trials)	Selected hyperparameters
Logistic regression	0.7687 (C=1, balanced)	0.7619	**0.7838** (trial 7)	C = 0.0419, class_weight=None
Random forest	0.7974 (500 trees, leaf 3, balanced)	0.7962	0.7970 (trial 10)	348 trees, max_depth 20, min_split 6, min_leaf 3, sqrt features, balanced_subsample

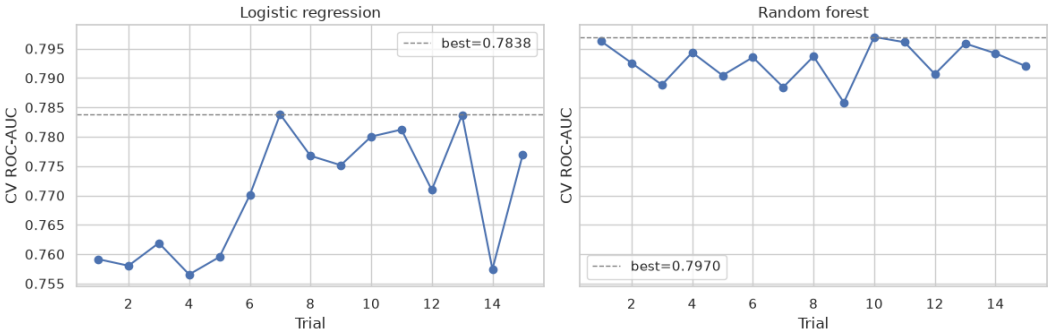


Figure 2 – CV ROC-AUC by trial, taken from the executed notebook (cell 15). Logistic regression climbs once BO leaves the over-fitting region (large C); the forest stays within a 0.011 band.

3.6 Why logistic regression gained and the forest did not

Logistic regression: the gain is regularisation. scikit-learn minimises $C \cdot \Sigma \ell_i + \tfrac{1}{2}\|w\|^2$, which is the negative log-posterior of a Gaussian prior $w \sim N(0, C \cdot I)$. Dividing through shows this is exactly MAP estimation:

$$\hat{w}_{\text{MAP}} = \arg\min_{w} \sum_{i=1}^{n} \ell(y_i,\ w^\top x_i) + \frac{1}{2C}\|w\|_2^2 \quad \Longleftrightarrow \quad w \sim \mathcal{N}(0,\ C\,I)$$

The original C = 1 corresponds to a prior standard deviation of 1.0 on each coefficient; the BO optimum C = 0.042 corresponds to ≈ 0.20 († $\sqrt{0.0419} = 0.205$). After one-hot encoding the model has **61 input columns** for 800 training rows and only 240 defaulters, so unshrunken coefficients over-fit. Heavier shrinkage trades a little bias for a larger variance reduction, which is exactly the regime where the bias–variance trade-off pays. The trajectory in Figure 2 confirms it: every LR trial with C ≥ 6 scored ≤ 0.762, while every trial with C ≤ 0.15 scored ≥ 0.775.

Random forest: already near its ceiling. Averaging many decorrelated trees controls variance by construction, so performance is flat across depth, leaf size and feature subsampling. The 15 forest trials span 0.7858–0.7970, a range of 0.011, whereas the fold-to-fold standard deviation of CV AUC in the original run was 0.039 (standard error of the five-fold mean ≈ 0.039/√5 = 0.017†). Differences smaller than that are noise, so the forest's 0.7974 → 0.7970 should be read as *no change*, and so should its holdout result.

> **Caveat to raise before an interviewer does**
>
> The reported 0.7838 and 0.7970 are the *maximum* over 15 noisy evaluations on the same folds that chose them, so they are optimistically biased (winner's curse). The unbiased check is the untouched holdout, where the forest scored 0.801 either way. A stricter protocol is **nested CV**: an inner loop tunes θ, an outer loop scores the tuned pipeline. Differences of this size are also hard to separate from fold-to-fold noise.

4. Improvement 2 – Uncertainty on the holdout ROC-AUC

The original reports a single holdout AUC of 0.801. With 200 applicants and just 60 defaulters, that number carries real sampling error. AUC is a rank statistic: it equals the probability that a randomly chosen bad applicant is scored above a randomly chosen good one (the normalised Mann–Whitney U):

$$\text{AUC} = \frac{1}{n_1 n_0} \sum_{i:\ y_i = 1} \sum_{j:\ y_j = 0} \left(\mathbf{1}[s_i > s_j] + \frac{1}{2}\mathbf{1}[s_i = s_j] \right)$$

Because the estimate is built from only 60 × 140 = 8,400 pairs, the notebook uses a **stratified percentile bootstrap**: resample goods and bads separately (keeping 140/60), recompute AUC 3,000 times, and take the 2.5% and 97.5% quantiles. Stratifying matters because an ordinary bootstrap can draw a different number of defaulters each time, adding variance unrelated to the model.

```
rng = np.random.default_rng(RANDOM_STATE)
y_arr, p_arr = y_test.to_numpy(), np.asarray(probabilities)
bad_idx, good_idx = np.flatnonzero(y_arr == 1), np.flatnonzero(y_arr == 0)
boot_auc = []
for _ in range(3000):
    sampled = np.concatenate([rng.choice(good_idx, size=len(good_idx), replace=True),
                              rng.choice(bad_idx,  size=len(bad_idx),  replace=True)])
    boot_auc.append(roc_auc_score(y_arr[sampled], p_arr[sampled]))
auc_low, auc_high = np.quantile(boot_auc, [0.025, 0.975])
```

Listing 2 – Stratified bootstrap (cell 21, part 1). Output: Holdout AUC 0.801; 95% interval 0.730–0.864; 60 defaulters of 200.

Independent cross-check. † The analytic Hanley–McNeil standard error for AUC = 0.801 with n_1 = 60 and n_0 = 140 is 0.037, giving a normal interval of 0.728–0.874. This agrees closely with the bootstrap (0.730–0.864), which is reassuring; the bootstrap interval is slightly asymmetric towards the lower side, as expected for a statistic bounded above by 1. **Interpretation:** the interval spans roughly ±0.07, which is larger than *every* difference between the two notebooks' holdout metrics. This is the statistical reason the holdout cannot separate them.

5. Improvement 3 – Repeated CV and a paired model comparison

The original picked the forest from one 5-fold run (0.797 vs 0.769). The improved notebook re-evaluates both tuned families on all 1,000 rows using `RepeatedStratifiedKFold(5 folds × 5 repeats)` with the *same* 25 splits for both models, and tests the per-split AUC differences $d_i = AUC_{RF,i} - AUC_{LR,i}$. Pairing removes the large between-split difficulty variation, because both models face identical folds.

```
repeat_cv = RepeatedStratifiedKFold(n_splits=5, n_repeats=5, random_state=RANDOM_STATE)
fold_auc = {fam: cross_val_score(make_pipeline(fam, params, prep), X, y,
                                 cv=repeat_cv, scoring="roc_auc", n_jobs=1)
            for fam, params in [("Random forest", forest_params), ("Logistic regression", logistic_params)]}
paired_gap  = fold_auc["Random forest"] - fold_auc["Logistic regression"]
paired_test = ttest_rel(fold_auc["Random forest"], fold_auc["Logistic regression"])
```

Listing 3 – Paired repeated-CV comparison (cell 21, part 2).

Quantity (reported)	Value
Random forest mean AUC (SD)	0.795 (0.029)
Logistic regression mean AUC (SD)	0.787 (0.033)
Mean paired gap, forest − logistic	0.008
Paired t-test p-value (naive)	0.012
Naive 95% interval for the gap	0.002 to 0.014

The statistical catch. The 25 differences are *not* independent: any two splits overlap heavily in training data, so the naive standard error s/√25 is too small. The notebook itself notes the interval is “descriptive”. The standard repair is the **Nadeau–Bengio corrected resampled t-test**, which inflates the variance by the ratio of test to training size:

$$\widehat{\mathrm{Var}}(\bar{d}) = \left(\frac{1}{J} + \frac{n_{\text{test}}}{n_{\text{train}}}\right) s_d^2, \qquad t = \frac{\bar{d}}{\sqrt{\left(\frac{1}{J} + \frac{n_{\text{test}}}{n_{\text{train}}}\right) s_d^2}} \sim t_{J-1}$$

† Here J = 25 splits and n_{test}/n_{train} = 200/800 = 0.25, so the variance inflation factor is (1/25 + 0.25)/(1/25) = 7.25 and the standard error inflates by √7.25 ≈ 2.69. Starting from the reported p = 0.012 (t ≈ 2.72 on 24 df) the corrected statistic is t ≈ 1.01, giving **p ≈ 0.32**, and the corrected 95% interval for the gap is roughly −0.009 to +0.025, which comfortably contains zero. (These are back-calculated from rounded published figures, so treat them as approximate.)

Take-away. Once the dependence between resamples is respected, the forest is *not* demonstrably better than the tuned logistic regression. The forest still has a nominal edge (0.795 vs 0.787 AUC), but in a regulated credit setting an interpretable, well-regularised logistic model with a statistically indistinguishable AUC is often the better deployment choice. That nuance is a strong discussion point.

Second caveat: the hyperparameters were tuned on the 800 training rows but are scored here on all 1,000; the effect is small, and nested CV removes it.

6. Improvement 4 – Class weighting versus calibration

Class weights change what the model optimises. If positives get weight w_1 and negatives w_0, the fitted model estimates posterior odds under a *re-weighted* population, not the real one. With "balanced" weights, $w_1/w_0 = n_0/n_1$ (here 700/300 = 7/3), and the learned odds are scaled by that factor:

$$\frac{\tilde{p}}{1-\tilde{p}} = \frac{w_1}{w_0} \cdot \frac{p}{1-p} \quad \Longrightarrow \quad p = \frac{\tilde{p}}{\tilde{p} + (1-\tilde{p})\,(w_1/w_0)}$$

A strictly increasing transform of the score cannot change the ranking, so **ROC-AUC is invariant** to class weighting, while **Brier score and mean predicted probability are not**. This is exactly what the notebook's experiment shows: holdout AUC is 0.806 without weights and 0.799 with them (a difference well inside the bootstrap interval), but the mean predicted P(bad) is **0.309** without weights (observed bad rate 0.300) against **0.428** with balanced weights, a clear upward bias of probabilities.

```
unweighted_params = dict(best_params); unweighted_params["class_weight"] = None
balanced_params   = dict(best_params); balanced_params["class_weight"]   = "balanced"
for label, fitted in [("No class weights", make_pipeline(best_name, unweighted_params, prep).fit(X_train, y_train)),
                      ("Balanced weights", make_pipeline(best_name, balanced_params,   prep).fit(X_train, y_train))]:
    p = fitted.predict_proba(X_test)[:, 1]
    print(f"{label}: holdout AUC={roc_auc_score(y_test, p):.3f}; "
          f"mean predicted P(bad)={p.mean():.3f}; observed bad rate={y_test.mean():.3f}")
```

Listing 4 – Weighting experiment (cell 21, part 3). Output: 0.806 / 0.309 (no weights) vs 0.799 / 0.428 (balanced).

† **Verification of the theory.** On a synthetic 5,000-row problem with 28.5% prevalence, an (almost) unregularised logistic regression gave mean predicted P(bad) = 0.285 unweighted and 0.437 with balanced weights; applying the formula above with $w_1/w_0 = (1-\pi)/\pi$ pulled the weighted model back to 0.286. AUC was 0.8134 in all three cases and Brier score returned from 0.1771 to 0.1507 (unweighted: 0.1507). For a forest the relationship is only approximate, because leaf probabilities are not a logistic model, so in production the correct step is an empirical recalibration (Platt or isotonic) fitted on held-out data.

Why it matters downstream. Expected-loss calculations in IFRS 9 / CECL-style work (the background material for this exercise) use the predicted probability itself as a probability of default, so a model whose average output is 43% when the true rate is 30% would overstate provisions. Ranking quality (AUC) and probability quality (calibration) must be reported separately. This also explains the SHAP base value of 0.5 seen in Section 10.

7. Improvement 5 – Stability of the "three most important features"

The task asks for the top three features. A ranking from one 200-row holdout can be fragile, so the improved notebook repeats permutation importance inside each of five outer folds. For feature j, importance is the mean drop in AUC when column j is shuffled R times, breaking its link with the target while keeping its marginal distribution:

$$\mathrm{PI}_j = \frac{1}{R}\sum_{r=1}^{R} \left[\mathrm{AUC}(\mathcal{D}) - \mathrm{AUC}(\mathcal{D}^{(j,r)})\right]$$

```
for fold_no, (tr_idx, va_idx) in enumerate(StratifiedKFold(5, shuffle=True, random_state=RANDOM_STATE).split(X, y),
    1):
    fold_model = make_pipeline(best_name, best_params, prep).fit(X.iloc[tr_idx], y.iloc[tr_idx])
    fold_perm = permutation_importance(fold_model, X.iloc[va_idx], y.iloc[va_idx],
                                       scoring="roc_auc", n_repeats=10, random_state=RANDOM_STATE + fold_no)
    order = np.argsort(fold_perm.importances_mean)[::-1]
    fold_top_features.append(X.columns[order[:3]].tolist())       # then count how often each feature appears
```

Listing 5 – Fold-wise permutation importance (cell 21, part 4).

Feature	Folds in top three (of 5)	Comment
checking_status	**5**	Rank 1 in every fold: a robust driver
duration	**4**	Stable
savings_status	3	Competes with credit_amount for the third slot
credit_history	1	
other_payment_plans	1	

Feature	Folds in top three (of 5)	Comment
credit_amount	1	Third on the single holdout, but top-three in only 1 fold

Reading. *checking_status* is unambiguous (holdout AUC drop 0.132 ± 0.035, about eight times any other feature), *duration* is stable, and the third place is genuinely contested. In the holdout runs the order of *duration* and *credit_amount* even swapped between notebooks (duration 0.0174 vs credit_amount 0.0155 originally; 0.0155 vs 0.0161 after tuning), with standard deviations of 0.007–0.010 that exceed the gaps. Reporting selection frequency is therefore more honest than reporting a rank. Permutation importance is also biased when features are correlated, as they are among one-hot-encoded credit attributes.

8. Improvement 6 – Age-group fairness and proxy audit

A credit model trained on age-correlated data can disadvantage young applicants. The notebook audits one specific, defensible criterion, **false-positive-rate parity** (a component of equalised odds): among applicants who are actually *good*, how often does the model wrongly flag them as bad?

$$\mathrm{FPR}_g = P(\hat{Y} = 1 \mid Y = 0,\ G = g), \qquad \Delta_{\mathrm{FPR}} = \mathrm{FPR}_{\text{under 25}} - \mathrm{FPR}_{\text{over 40}}$$

```
audit = X_test[["age"]].copy()
audit["actual_good"], audit["predicted_bad"] = (y_test == 0).to_numpy(), (predictions == 1)
audit["age_group"] = np.select([audit["age"] < 25, audit["age"] > 40], ["Under 25", "Over 40"], default="25-40")
for group in ["Under 25", "Over 40"]:
    subset = audit[(audit["age_group"] == group) & audit["actual_good"]]
    fpr = subset["predicted_bad"].mean()
```

Listing 6 – Group false-positive rates (cell 21, part 5).

Age group	Good applicants	False positives	FPR	† Wilson 95% interval
Under 25	20	7	35.0%	18.1% – 56.7%
Over 40	51	8	15.7%	8.2% – 28.0%

The gap is 19.3 percentage points, a ratio of about 2.2×. † **But the sample is small:** a Fisher exact test on 7/20 versus 8/51 gives p ≈ 0.105, and the Wilson intervals overlap. The honest conclusion is “meaningful disparity signal, not statistically conclusive on 200 rows”, which justifies monitoring in production rather than a verdict.

Proxy diagnostic. Removing the *age* column does not remove age information if other fields encode it (employment length, housing, job). The notebook trains a forest to predict “under 25” from all fields *except* age; a high cross-validated AUC means age is recoverable. The result was **AUC 0.807 (fold SD 0.024)**, so age is substantially reconstructible and simply dropping the column would not make the model age-blind. (Whether and how age may be used is also a legal question that varies by jurisdiction, which is why this notebook only diagnoses and does not prescribe.)

9. Improvement 7 – Prediction-time input validation

This responds directly to the job description's emphasis on ML Ops and on working with data-engineering teams. A fitted scikit-learn pipeline is quietly permissive: `OneHotEncoder(handle_unknown='ignore')` turns an unseen category into an all-zero row, and a numeric field out of range is simply standardised. Both produce a confident-looking score from invalid input, a classic **training/serving skew** failure. The fix is a *data contract* enforced at the boundary, failing fast with row- and field-level messages:

```
NUMERIC_BOUNDS = {"duration": (1, 120), "credit_amount": (1, 1_000_000), "installment_commitment": (1, 4),
                  "residence_since": (1, 4), "age": (18, 100), "existing_credits": (1, 10), "num_dependents": (1, 20)}
CATEGORY_LEVELS = {col: set(df[col].dropna().astype(str).unique()) for col in cat_cols}

def validate_applicants(applicants):
    ...  # type check -> exact column set -> no nulls -> numeric integer-in-bounds -> known category levels
    if issues: raise ValueError("Applicant validation failed:\n- " + "\n- ".join(issues))
    return checked

def score_applicants(applicants, fitted_model=best):      # validation ALWAYS precedes predict_proba
    return fitted_model.predict_proba(validate_applicants(applicants))[:, 1]
```

Listing 7 – Guarded scoring interface (cell 21, part 6). Demonstrated rejection: “age: expected integer values in [18, 100] at rows [30]”.

Design points to mention: the bounds are *operational guardrails*, deliberately wider than the training range (training ages are 19–75), and should be agreed with the business; unseen categories raise an error rather

than being silently zero-encoded; and the same function is the natural unit-test target for a CI pipeline (parametrised tests for each rejection path). The notebook text also tells users to call `score_applicants`, never `predict_proba`, directly.

10. Improvement 8 – SHAP explanations

10.1 Theory: Shapley values

Permutation importance says which columns *matter to accuracy*; SHAP says how each feature *moved an individual score*. SHAP treats the prediction as a payout in a cooperative game among M features and assigns each feature its Shapley value:

$$\phi_j = \sum_{S \subseteq N \setminus \{j\}} \frac{|S|!\ (M - |S| - 1)!}{M!} [v(S \cup \{j\}) - v(S)], \qquad f(x) = \phi_0 + \sum_{j=1}^{M} \phi_j$$

Here v(S) is the expected model output when only the features in S are fixed to the applicant's values. It is the *unique* attribution satisfying **efficiency** (contributions sum exactly to f(x) − φ_0), **symmetry**, **dummy** (an unused feature gets zero) and **additivity**. Exact computation needs 2^{M-1} coalitions per feature, i.e. intractable for 61 encoded columns. **TreeSHAP** (Lundberg et al.) exploits tree structure, routing absent features down both branches weighted by training cover and using dynamic programming along each path, to compute the same values in polynomial time.

10.2 Aggregating one-hot dummies back to raw fields

The forest sees 61 encoded columns, but stakeholders think in 20 raw fields. The notebook maps each encoded name to its source field (longest-prefix match on `get_feature_names_out()`) and sums within groups:

$$\phi_{i,g}^{\text{field}} = \sum_{j \in g} \phi_{i,j}, \qquad \phi_0 + \sum_g \phi_{i,g}^{\text{field}} = f(x_i) \quad \text{(efficiency preserved)}$$

```
for col in raw_features:
    idx = np.where(orig_of == col)[0]
    aggregated[col] = shap_mat[:, idx].sum(axis=1)            # one signed column per raw field
agg_importance = np.abs(pd.DataFrame(aggregated)).mean(axis=0).sort_values(ascending=False)
```

Listing 8 – Aggregation to original fields (cell 27).

† **Verification by brute force.** I enumerated all Shapley coalitions for a 4-feature toy model (one numeric driver, two dummies from one categorical field, one interacting numeric). Local accuracy held exactly (base 0.0457 + Σφ = 1.0200 = f(x)); the grouped contribution $\varphi_1 + \varphi_2 = 0.1885$ preserved the identity; and treating the field as a single player in a 3-player game gave the same 0.1885. A nuance: summing dummies is the standard convention and always preserves efficiency, but the Shapley value of the *group as one player* can differ from the sum when the dummies interact with other features. Note also that the importance is the mean of the absolute *aggregated* value, not the sum of absolute dummy values, which would double-count levels that offset each other.

10.3 Results

Rank	Raw field	Mean |aggregated SHAP|	Permutation importance, holdout AUC drop
1	checking_status	0.102	0.132 (rank 1, improved run)
2	savings_status	0.038	not in holdout top 3 (0.012 in the original run)
3	duration	0.038	0.0155 (rank 3, improved run)
4	credit_history	0.028	—
5	other_payment_plans	0.025	—
6	credit_amount	0.023	0.0161 (rank 2, improved run)

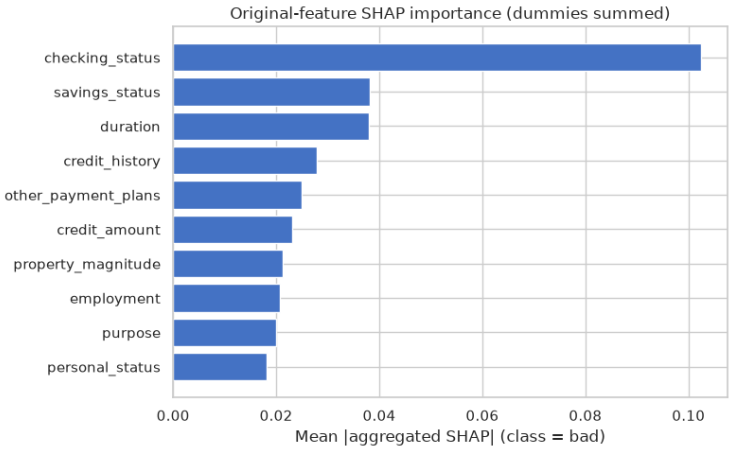


Figure 3 – Aggregated SHAP importance by original field, from the executed notebook (cell 27).

The improved notebook printed only its top three permutation values, hence the dashes. Both methods agree that *checking_status* dominates, and duration and credit_amount sit in the top six of both. SHAP gives *savings_status* a higher rank than permutation does, consistent with the fold-stability result in Section 7, where savings_status is the contested third place. Disagreement between attribution methods on the lower ranks is itself information about how little the data can say.

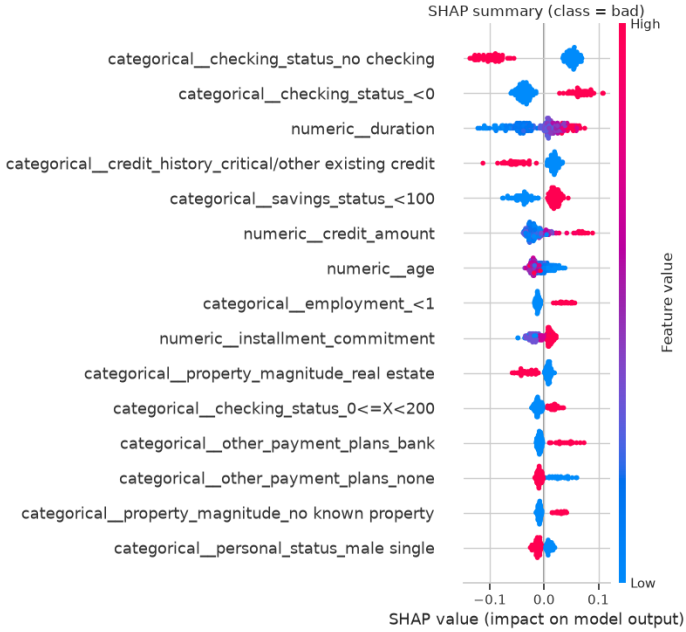


Figure 4 – SHAP beeswarm for class bad (encoded features), from the executed notebook (cell 29).

Two details a careful reviewer will probe. (i) The SHAP base value printed is **0.5**, not the 30% default rate. That is a direct consequence of `class_weight='balanced_subsample'`: the forest's leaf probabilities live on the re-weighted scale from Section 6, so SHAP explains *that* output. (ii) The local-accuracy check returned a mean absolute residual of 0.0031 between base + Σφ and predict_proba, small but not machine-zero, as path-dependent TreeSHAP uses training-cover expectations rather than the exact mean prediction. Neither invalidates the explanations; both should be stated.

11. Honest assessment and next steps

Dimension	Verdict	Evidence
Predictive performance of the champion	**No change**	Holdout AUC 0.801 both; interval ±0.07; forest insensitive to tuning
Baseline model quality	**Improved**	LR CV AUC 0.769 → 0.784; MAP/shrinkage explanation
Statistical rigour	**Much improved**	Bootstrap CI, repeated paired CV, rank stability; corrected test shows forest vs LR gap not significant
Probability quality awareness	**Improved**	Weighting changes mean P(bad) 0.309 → 0.428 while AUC is flat; no recalibration yet
Explainability and governance	**New**	SHAP (local + global), fairness FPR audit, proxy test, input contract
Operational readiness	**Partial**	Validation layer exists; no tests, packaging, model registry or monitoring yet

Suggested next increments (good material for live pair programming)

- **Nested CV** around the BO loop to remove the winner's-curse bias, and the Nadeau–Bengio test as a reusable function.
- **Recalibrate probabilities** (isotonic or Platt on out-of-fold predictions), then report reliability curves, Brier decomposition and expected calibration error.
- **Cost-sensitive threshold** chosen from an explicit loss matrix instead of the default 0.5, and re-run the FPR audit at that threshold.
- **CI/CD hooks**: pytest cases for every rejection path of `validate_applicants`, a smoke test that scoring a known row reproduces a stored probability, and a pre-commit check that notebooks execute top to bottom.
- **ML Ops**: log each BO trial and final model to MLflow (or equivalent), version the data schema alongside the model, and add drift monitors (PSI) on the validated input fields.
- **Business framing**: translate checking_status, duration and credit_amount into plain-language adverse-action reasons using the aggregated SHAP outputs.

Likely questions and short answers

Question	Answer in one breath
Why Bayesian optimisation rather than grid search?	Each evaluation is a 5-fold refit of a noisy objective; a GP surrogate with EI concentrates evaluations where improvement is plausible. Here 15 trials per model gave a +0.022 AUC gain over random warm-up for LR.
Did tuning help the final model?	No: the forest was already flat across settings and the holdout is unchanged at 0.801. The gain was on the logistic baseline, via stronger L2 shrinkage.
Is the random forest really better than logistic regression?	Nominally (0.795 vs 0.787), but after the Nadeau–Bengio correction the gap is not significant ($p \approx 0.3$), so interpretability may justify the simpler model.
Why is AUC unchanged by class weights but probabilities are not?	Weights scale the odds by a constant, a monotone transform of the score: ranking is preserved, calibration is not.
Is the third most important feature reliable?	No. checking_status (5/5 folds) and duration (4/5) are stable; the third place rotates between savings_status, credit_amount and others.

Provenance. Notebook figures (Figures 2–4) are extracted from the executed notebook. Figure 1 and all † items are my own supplementary computations: the sandbox used to build this report had neither the CSV nor network access, so the notebooks were not re-run, and the numbers quoted are those printed in the saved outputs. Corrected-test, Hanley–McNeil, Wilson and Fisher figures are back-calculated from the reported rounded values.